

Lambdas and Streams: What, Not How



Did you know, you
don't have to write absolutely
everything yourself? You can
get the APIs to do the work
for you!

This is a "convenience factory method" for creating a new List from a known group of values. We saw this in Chapter 11.

```
List<String> allColors = List.of("Red", "Blue", "Yellow");  
for loop { for (String color : allColors) {  
            System.out.println(color);  
        }
```

For each item in the list create a temporary variable, color...

...then print out each color.

```
List<String> allColors = List.of("Red", "Blue", "Yellow");  
allColors.forEach(color -> System.out.println(color));
```

For each item in the list...

Create a temporary variable named color

Print out the color

java.util.stream.Stream

Stream<T> distinct()

Returns a stream consisting of the distinct elements

Stream<T> filter(Predicate<? super T> predicate)

Returns a stream of the elements that match the given predicate.

Stream<T> limit(long maxSize)

Returns a stream of elements truncated to be no longer than maxSize in length.

<R> Stream<R> map(Function<? super T,? extends R> mapper)

Returns a stream with the results of applying the given function to the elements of this stream.

Stream<T> skip(long n)

Returns a stream of the remaining elements of this stream after discarding the first n elements of the stream.

Stream<T> sorted()

Returns a stream of the elements of this stream, sorted according to natural order.

// more

(These are just a few of the methods in Stream... there are many more.)

These generics do look a little intimidating, but don't panic! We'll use the map method later, and you'll see it's not as complicated as it seems.

Assuming we had a List of Strings like this...

```
List<String> strings = List.of("I", "am", "a", "list", "of", "Strings");  
Stream<String> stream = strings.stream();
```

...we can call this method to get a Stream of these Strings.

```
Stream<String> limit = stream.limit(4);
```

Sets the maximum number of results to return to 4

The limit method returns another Stream of Strings, which we'll assign to another variable

File Edit Window Help SliceAndDice

```
%java LimitWithStream
```

```
limit = java.util.stream.SliceOps$1@7a0ac6e3
```

```
System.out.println("limit = " + limit);
```

java.util.stream.Stream

boolean anyMatch(Predicate<? super T> predicate)

Returns true if any element matches the provided predicate.

long count()

Returns the number of elements in this stream.

<R,A> R collect(Collector<? super T,A,R> collector)

Performs a mutable reduction operation on the elements of this stream using a Collector.

Optional<T> findFirst()

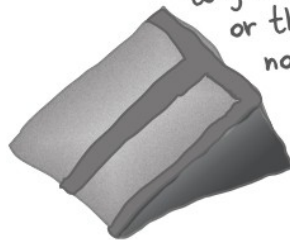
Returns an Optional describing the first element of this stream, or an empty Optional if the stream is empty.

// more

(These are some terminal operations on Stream.)

Yes, this looks even scarier than the map method! Don't panic, these generic types help the compiler, but you'll see when we actually use this method, we don't have to think about these generic types.

Call a "do it"
to get the cake,
or there is
no cake.




```
List<String> strings = List.of("I", "am", "a", "list", "of", "Strings");
```

```
Stream<String> stream = strings.stream();
```

```
Stream<String> limit = stream.limit(4);
```

```
long result = limit.count();
```

```
System.out.println("result = " + result);
```

Call the count terminal operator, and store the output in a variable called result

The stream contained Strings, so the output object will also contain Strings.

Terminal operation that will collect the output into some sort of Object.

This method returns a Collector that will output the results of the stream into a List.

```
List<String> result = limit.collect(Collectors.toList());
```

The toList Collector will output the results as a List.

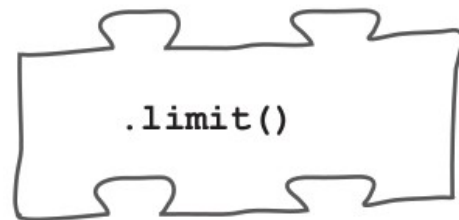
A helpful class that contains methods to return common Collector implementations.

```
System.out.println("result = " + result);
```

- ① Get the Stream from a **source** collection.



- ② Call zero or more **intermediate operations** on the Stream.



- ③ Output the results with a **terminal operation**.



```
List<String> strings = List.of("I", "am", "a", "list", "of", "Strings");
```

```
List<String> result = strings.stream()  
                        .limit(4)  
                        .collect(Collectors.toList());
```

Get the stream for the collection

Set a limit to return a maximum of 4 results from the stream

Formatted to align each operation directly underneath the one above, to clearly show each stage.

Returns the results of the operation as a List

```
System.out.println("result = " + result);
```



```
List<String> strings = List.of("I", "am", "a", "list", "of", "Strings");
```

```
List<String> result = strings.stream()
```

```
.sorted()
```

Limit the stream to just
four elements.

```
.limit(4)
```

```
.collect(Collectors.toList());
```

```
System.out.println("result = " + result);
```

Sort what's in the stream (not the original collection), using natural order, before limiting the results.

File Edit Window Help InChains

```
%java ChainedStream
```

```
result = [I, Strings, a, am]
```

Natural ordering of Strings will place capitalized Strings ahead of lowercase Strings.

Lambda expression that tells the sorted method how to sort the strings in the stream. This lambda expression represents a Comparator, which we talked about in the previous chapter. We'll recap lambdas later in this chapter.

```
List<String> result = strings.stream()  
    .sorted((s1, s2) -> s1.compareToIgnoreCase(s2))  
    .limit(4)  
    .collect(Collectors.toList());
```

This method from the String class compares the String with another String, in a way that ignores upper or lowercase.

File Edit Window Help IgnoreCaps

```
%java ChainedStream
```

```
result = [a, am, I, list]
```

```
List<String> result = strings.stream()  
    .sorted((s1, s2) -> s1.compareToIgnoreCase(s2))  
    .skip(2)  
    .limit(4)  
    .collect(Collectors.toList());
```

File Edit Window Help BoxersDolt

```
%java ChainedStream
```

```
result = [I, list, of, Strings]
```

← The stream skipped over the first two elements.

I only start my day
once I know exactly
what I'm going to do, and
exactly how to do it.

Streams are lazy



Terminal operations do all the work

Since intermediate operations are *lazy*, it's up to the terminal operation to do everything.

- 1 Perform all the intermediate operations as efficiently as possible. Ideally, just going through the original data once.
- 2 Work out the result of the operation, which is defined by the terminal operation itself. For example, this could be a list of values, a single value, or a boolean (true/false).
- 3 Return the result.

```
List<String> result = strings.stream()  
                             .sorted()  
                             .skip(2)  
                             .limit(4)  
                             .collect(Collectors.toList());
```

Terminal operation:

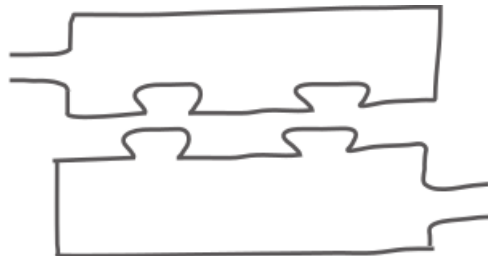
1. performs all intermediate operations, in this case: sort; skip; limit.
2. collects the results according to the instructions passed into it
3. returns those results

Collectors is a class that has static methods that provide different implementations of Collector. Look at the Collectors class to find the most common ways to collect up the results.

The collect method takes a Collector, the recipe for how to put together the results. In this case, it's using a helpful predefined Collector that puts the results into a List.

Guidelines for working with streams

- ① **You need at least the first and last pieces to create a stream pipeline.**



- ② **You can't reuse Streams.**

```
Stream<String> limit = strings.stream()  
                                .limit(4);  
List<String> result = limit.collect(Collectors.toList());  
List<String> result2 = limit.collect(Collectors.toList());
```

File Edit Window Help ClosingTime

```
%java LimitWithStream
```

```
Exception in thread "main" java.lang.IllegalStateException: stream has  
already been operated upon or closed  
    at java.base/java.util.stream.AbstractPipeline.  
evaluate(AbstractPipeline.java:229)
```

③ **You can't change the underlying collection while the stream is operating.**

If you do this, you'll see strange results, or exceptions. Think about it—if someone asked you a question about what was in a shopping list and then someone else was scribbling on that shopping list at the same time, you'd give confusing answers too.



I'm so confused!
I'm just trying to
read this list, but
it keeps changing!

Stream operations don't change the original collection.

```
List<String> strings = List.of("I", "am", "a", "list", "of", "Strings");

Stream<String> limit = strings.stream()
                              .limit(4)
                              .collect(Collectors.toList());

System.out.println("strings = " + strings);
System.out.println("result = " + result);
```

File Edit Window Help Untouchable

```
%java LimitWithStream
```

```
strings = [I, am, a, list, of, Strings]
result = [I, am, a, list]
```

No changes to original collection after the stream operations are run.

Only the output object has the results of the query. This is a brand new List.

Lambda expressions are objects, and you run them by calling their Single Abstract Method

A lambda expression implements a Functional Interface.

This is a placeholder type to give you an idea of what the method would look like. We'll look at specific Functional Interfaces throughout this chapter.

```
void forEach( SomeFunctionalInterface lambda ) {  
    for (Element element : list) {  
        lambda.singleAbstractMethodName(element);  
    }  
}
```

This would be the name of whatever is the Single Abstract Method in the functional interface.

"element" is the lambda's parameter, the "item" in the lambda expression on the last page.

Lambdas aren't magic;
they're just classes
like everything else.

Comparator Interface

```
public interface Comparator<T> {  
    int compare(T o1, T o2);  
}
```

Lambda expression (implements Comparator)

```
(s1, s2) -> s1.compareToIgnoreCase(s2)
```

Method arguments

The compiler sees the lambda body (compareToIgnoreCase) has an int result, and that matches the return type of the compare method on the Comparator interface.

```
(String s1, String s2) -> {  
    return s1.compareToIgnoreCase(s2);  
}
```

Anatomy of a lambda expression

The number, and types, of the parameters to the lambda expression are determined by the Functional Interface it implements.

The parameter types for the lambda expression are not required, but you can add them to be explicit. This may be required if there's more than one functional interface that might match.

(String s1, String s2) ->

If the lambda body is inside curly braces, you must put semicolons on the end of all the lines, just like lines in a normal Java method.

{
}

Lambda expressions can be wrapped in curly braces. If you have a lambda expression that's longer than one line, you **MUST** wrap it in curly braces.

return s1.compareToIgnoreCase(s2);

If the lambda overrides a method that returns a value, and the lambda body is inside curly braces, you need to put a return statement at the end of the lambda body. If the lambda expression is a single line, the compiler can work out what needs to be returned.

The lambda body, which is either a single line or multiple lines inside curly braces, is the core functionality. This is the code that would make up the body of the method if this was a fully fledged Java class implementing the functional interface. In this case, the lambda body is the logic for the compare() method in Comparator.

A lambda might have more than one line

Here's a lambda expression that implements `Comparator<String>` and results in the collection being sorted by string length in descending order.

```
(str1, str2) -> {  
    int l1 = str1.length();  
    int l2 = str2.length();  
    return l2 - l1;  
}
```

Semicolons required.

Curly braces required for multiline lambda expressions.

Return keyword required.

Life would be boring if we all looked the same.

No need for curly braces

```
(str1, str2) -> str2.length() - str1.length()
```

No need for "return"

No semicolons

A lambda might not return anything

Look! No round brackets! We'll see this again in a minute.

Multiline
lambda

```
str -> {  
    String output = "str = " + str;  
    System.out.println(output);  
}
```

No return value

```
@FunctionalInterface  
public interface  
Consumer<T> {  
    void accept(T t);  
}
```

Method is void on the
Functional Interface

A lambda might have zero, one, or many parameters

If a lambda expression doesn't take any parameters, you need to use empty brackets to show this.

```
() -> System.out.println("Hello!")
```

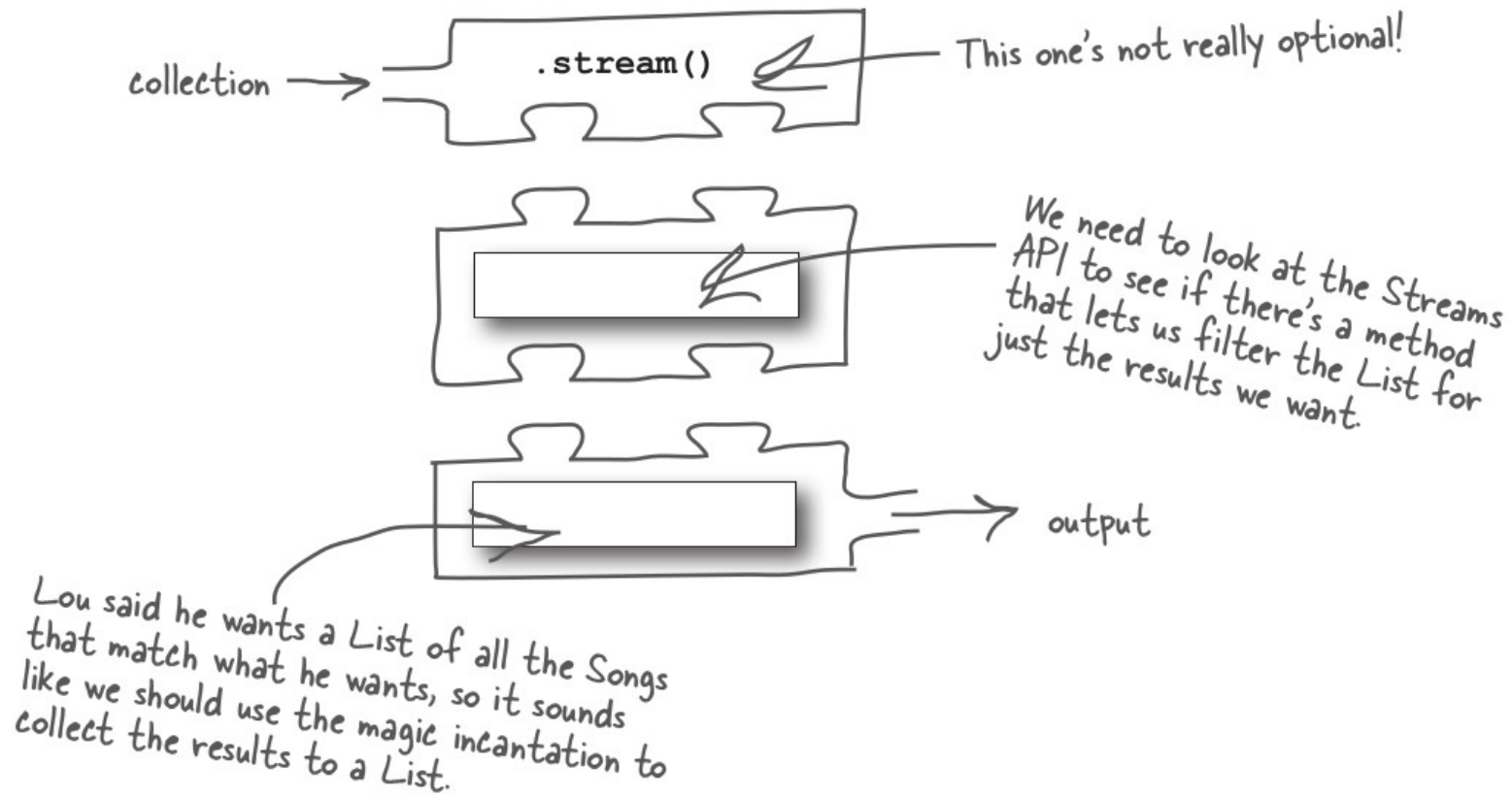
No method parameters

```
@FunctionalInterface  
public interface Runnable {  
    void run();  
}
```

Now that I have data about
what's been played on my
jukebox, I want to know more!



Lou's Challenge #1: Find all the "rock" songs



java.util.stream.Stream

Stream<T> filter(Predicate<? super T> predicate)

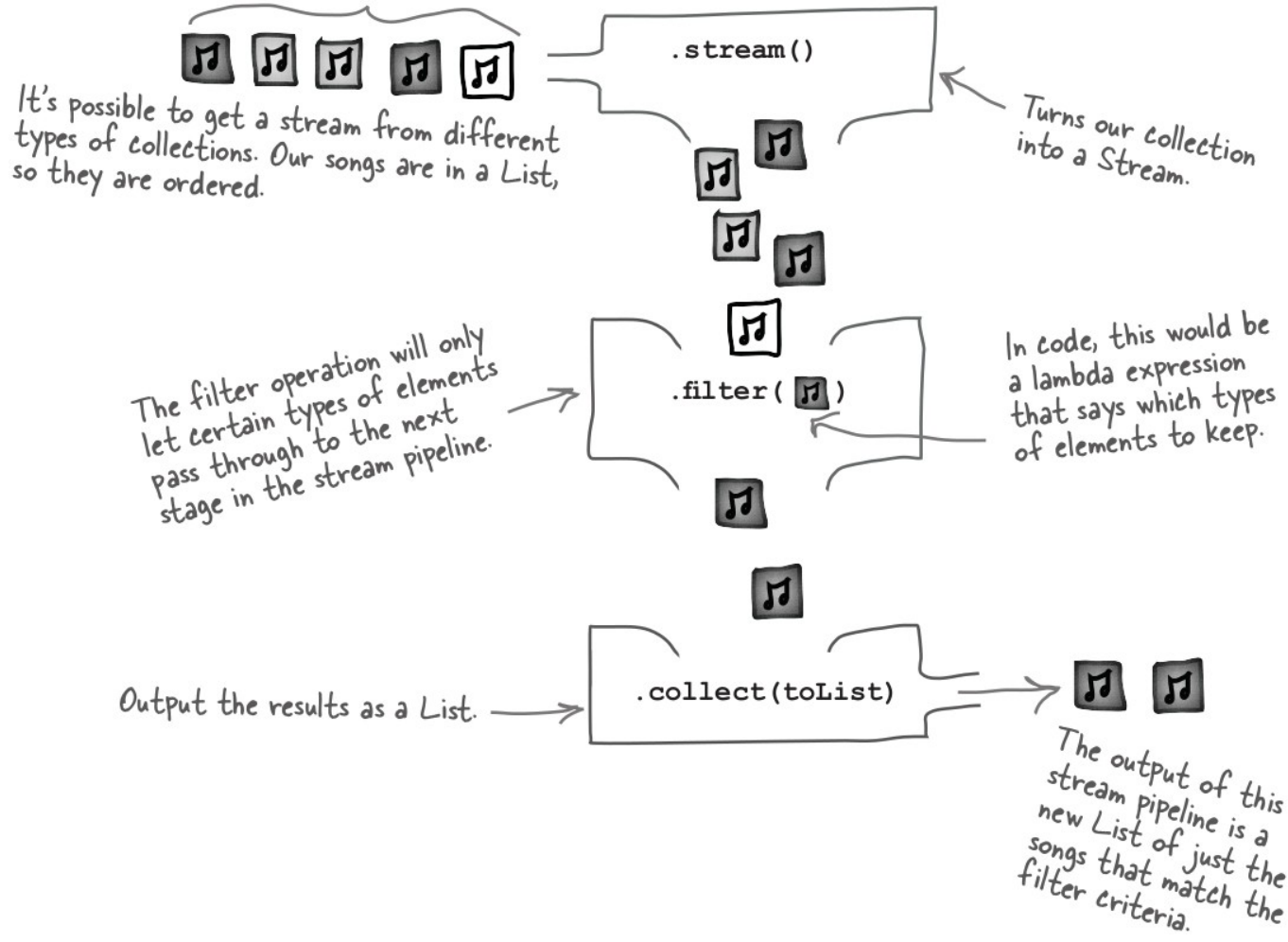
Returns a stream of the elements that match the given predicate.

<R,A> R collect(Collector<? super T,A,R> collector)

Performs a mutable reduction operation on the elements of this stream using a Collector.

Remember that for now we're just going to use the "magic incantation" to collect into a List.

These squares represent the songs; they're similar but not the same. Each different shade represents a different genre.



```
public class JukeboxStreams {
    public static void main(String[] args) {
        List<Song> songs = new Songs().getSongs();
```

This is a List of Songs.

This is a Song because
filter() is acting on a
Stream of Songs.

Get the genre (a String) from the
song and see if it's "Rock." This
will return a true or false.

The stream pipeline
will return a List
of Songs.

```
List<Song> rockSongs = songs.stream()
    .filter(song -> song.getGenre().equals("Rock"))
    .collect(Collectors.toList());
```

Puts the results into a List.

This lambda implements
Predicate.

```
System.out.println(rockSongs);
```

```
}
```

```
class Songs {
    // as Ready-Bake Code
```

```
class Song {
    // as Ready-Bake Code
```

File Edit Window Help StonefaceVimes

```
%java JukeboxStreams
```

```
[Cassidy, Grateful Dead, Rock
With a Little Help from My Friends, The Beatles, Rock,
Come Together, Ike & Tina Turner, Rock,
With a Little Help from My Friends, Joe Cocker, Rock,
Immigrant song, Led Zeppelin, Rock]
```

```
List<Song> rockSongs = songs.stream()
```

```
.filter(song -> song.getGenre().contains("Rock"))  
.collect(Collectors.toList());
```

Returns true if the genre
has the word "Rock" in it
anywhere

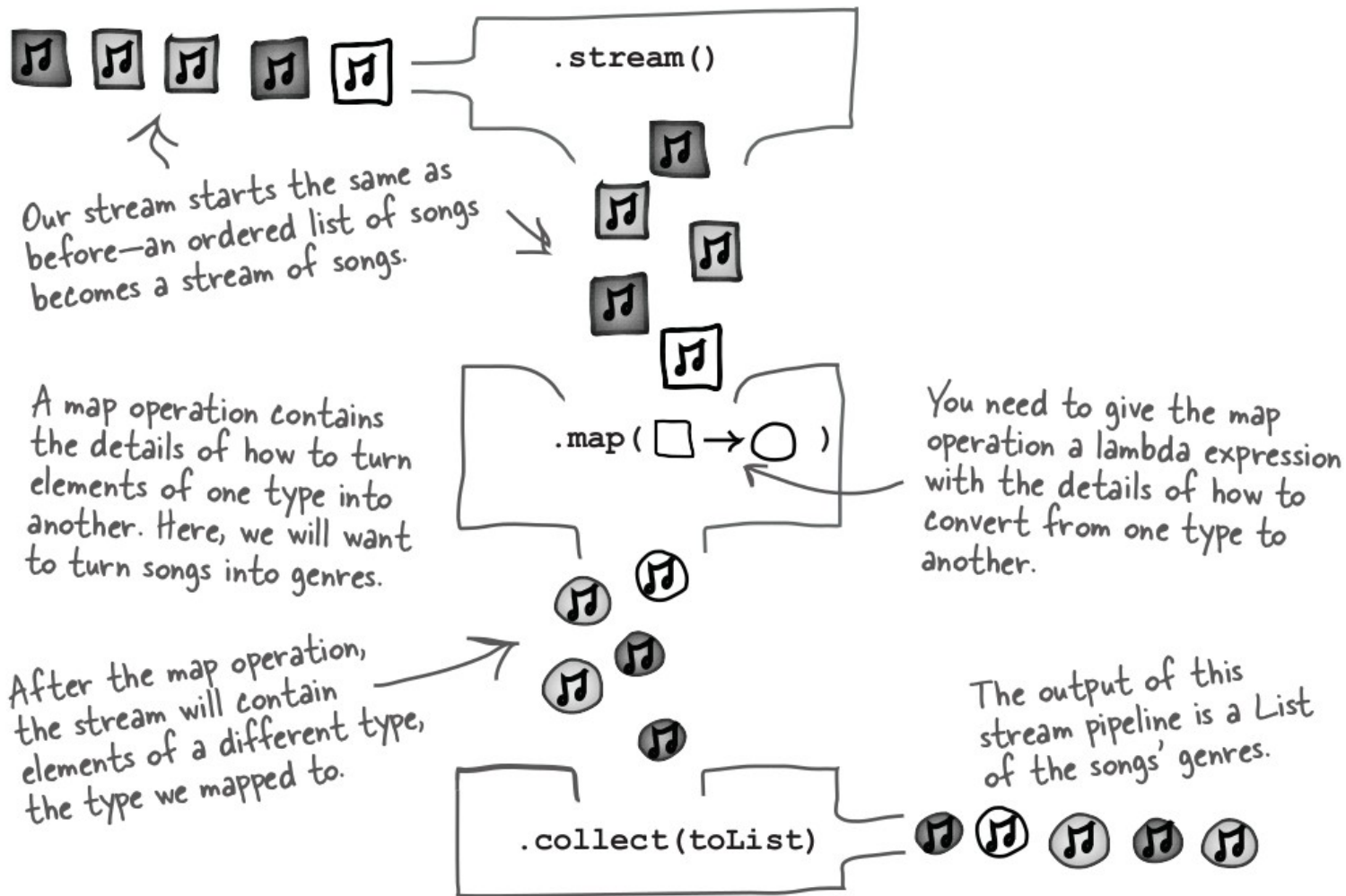
File Edit Window Help YouRock

```
%java JukeboxStreams
```

```
[Cassidy, Grateful Dead, Rock  
50 ways, Paul Simon, Soft Rock  
Hurt, Nine Inch Nails, Industrial Rock  
Hurt, Johnny Cash, Soft Rock  
...
```

Now the stream returns different
types of rock songs.

Lou's Challenge #2: List all the genres



Returns some Object

```
@FunctionalInterface  
public interface Function<T, R> {  
    R apply(T t);  
}
```

Takes a single parameter

The result will be a List of Strings, because genre is a String.

This is a List of Song objects.

A single parameter, a Song because this map() is on a Stream of Songs.

```
List<String> genres = songs.stream()  
    .map(song -> song.getGenre())  
    .collect(toList());
```

Puts the results into a List

The lambda body can return an object of any type. By calling getGenre on the song, the stream after this point will be a stream of (genre) Strings.

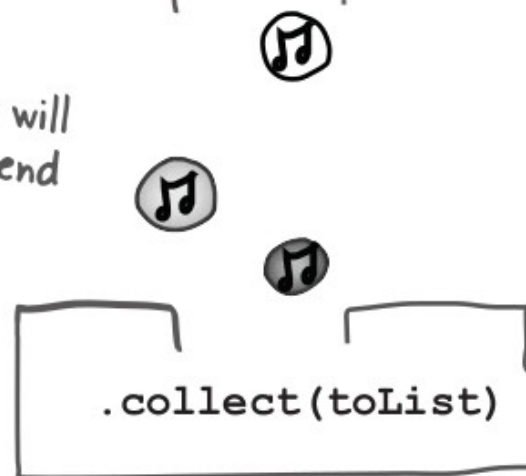
Removing duplicates

The distinct operation will stop any duplicate elements getting through.



This intermediate operation doesn't take any arguments; it's just a command on its own.

Only one of each object will make it through to the end of the pipeline.



The output of this stream pipeline is a new List of the unique genres.

```
List<String> genres = songs.stream()
    .map(song -> song.getGenre())
    .distinct()
    .collect(Collectors.toList());
```

Having this in the stream
pipeline means there will be no
duplicates after this point

Outputs a much more
readable list of all the genres

File Edit Window Help UniquelsGood

```
%java JukeboxStreams
```

```
[Electronic, R&B, Rock, Soft Rock,  
Industrial Rock, Alternative Rock,  
Blues rock, Pop, Latin]
```

```
String songTitle = "With a Little Help from My Friends";  
List<String> result = allSongs.stream()  
    .filter(song -> song.getTitle().equals(songTitle))  
    .map(song -> song.getArtist())  
    .filter(artist -> !artist.equals("The Beatles"))  
    .collect(Collectors.toList());
```

```
Set<String> genres = songs.stream()
```

```
    .map(song -> song.getGenre())  
    .collect(Collectors.toSet());
```

Store the results in a Set of Strings, not a List. Sets cannot contain duplicates.

Put the results into a Set, which will automatically only have unique entries.

```
boolean result =  
    songs.stream()  
        .anyMatch(s -> s.getGenre().equals("R&B"));
```

```
boolean anyMatch(Predicate p);  
boolean allMatch(Predicate p);  
boolean noneMatch(Predicate p);
```

```
Optional<Song> result =  
    songs.stream()  
        .filter(s -> s.getYear() == 1995)  
        .findFirst();
```

```
Optional<T> findAny();  
Optional<T> findFirst();  
Optional<T> max(Comparator c);  
Optional<T> min(Comparator c);  
Optional<T> reduce(BinaryOperator a);
```

```
long result =  
    songs.stream()  
        .map(Song::getArtist)  
        .distinct()  
        .count();
```

```
long count();
```



Wait a minute. How can a result be "Optional"? What does that even mean?

Well, some operations may return something, or may not return anything at all

Optional is a wrapper

```
Optional<IceCream> optional =  
    getIceCream("Strawberry");
```

```
if (optional.isPresent()) {  
    IceCream ice = optional.get();  
} else {  
    System.out.println("No ice  
cream for you!");  
}
```

Strawberry ice
cream, please!

Here you go!



Do you have
something
inside?

Yes.



Great! Give it to
me! Please.

All yours.

